

INSTITUTO POLITÉCNICO NACIONAL

ESCUELA SUPERIOR DE COMPUTO

Bucles Y Arrays

14/Octubre/2024

Practica 4

Alumno:

Guzmán Celaya Fernando

Grupo: 1CV1

Unidad de aprendizaje:

Fundamentos De Programación

Desarrollo.

Se escribió un programa realizado anteriormente en clase donde este código recibía un número se validaba si era positivo o no y se obtiene la raíz cuadrada de este número haciendo uso de la librería `math.h`, también se realizó un bucle para solicitar constantemente un valor numérico en caso de que no se cumpliera la condición de que fuera positivo.

```
// Includes
#include <stdio.h>
#include <math.h>

// Main Function
Tabnine | Edit | Test | Explain | Document | Ask
int main()
{
    // Variables
    float x;
    do
    {
        // Input
        printf("Dame un numero positivo");
        scanf("%f", &x);
        // Condition
        if (x < (float)0)
        {
            // Error
            printf("Te dije que numero positivo");
        }
    } while (x < (float)0);

    // Output
    printf("La raiz del numero: %.4f es: %.4f", x, sqrt(x));
}
```

Finalmente se ejecuto el programa probando dos situaciones, una cuando se ingresa un valor positivo y otra cuando se ingresa un valor negativo y se requiere que el usuario ingrese otro valor.

```
escom@L-1105-24 MINGW64 /e/Programming/Codes
$ gcc positive.c -o ./compiled/root.out

escom@L-1105-24 MINGW64 /e/Programming/Codes
$ ./compiled/root.out
Dame un numero positivo16
La raiz del numero: 16.0000 es: 4.0000
```

```

escom@L-1105-24 MINGW64 /e/Programming/Codes
● $ ./compiled/root.out
Dame un numero positivo:-35
Te dije que numero positivo
Dame un numero positivo:-23
Te dije que numero positivo
Dame un numero positivo:35
La raiz del numero: 35.0000 es: 5.9161

```

Posteriormente se vio el código que se había planteado el pseudocódigo anteriormente (unas clases atrás), comenzado con el ingreso de los datos y la validación de estos datos, finalmente se ejecuto y probaron estas validaciones.

```

#include <stdio.h>

int main() {
    long int n;
    char cin[10], c;

    char tabla[23] = "TRWAGMYFPDXBNJZSQVHLCKE";

    printf("Escribe el entero: ");
    scanf("%ld", &n);

    printf("Pusiste: %ld\n", n);

    if (n <= 0 || n >= 1000000000) {
        printf("%ld no tiene 8 dígitos\n", n);
        return (1);
    }

    /*else{
        printf("%ld esta en el rango\n", n);
        return(0);
    }*/
}

```

```

printf("Escribe la letra: ");
scanf("%s", cin);
c = cin[0];

if (c == 'i' || c == 'o' || c == 'u' || c == 'I' || c == 'O' || c == 'U') {
    printf("Caracter %c no valido.\n", c);
    return (1);
}

printf("Pusiste: %c\n", c);

// Verificacion
int modulo = n % 23;
if (c == tabla[modulo]) {
    printf("Código valido\n");
} else {
    printf("Código no valido\n");
}

```

Se ejecuto el programa usando el código 23 que al realizar la operación de modulo da 0 y esta letra en lo establecido es T y se mando esa misma letra por lo que el resultado de compararlo es correcto (True)

```

+52 1 55 3674 6387

2s © gcc Practica4.c -o practica4.out

Matemáticas admon 1 2024_2
+52 1 55 4572 5040 Un rato no más

© ./practica4.out
Escribe el entero: 23
Pusiste: 23
Escribe la letra: T
Pusiste: T
Código valido

+52 1 55 8440 1766
+52 1 55 3263 9190
FCE CERTIFICATION

```

Por otra parte, separe el funcionamiento en distintas funciones donde se crearon 2 funciones para validar los datos ingresados y una que realizara la operación para “decodificar” la letra del código y comprobar si es la correcta.

```
// NOTE Este codigo recibe un caracter (letra) y un codigo numerico, realiza
// la operacion modulo y con el resultado se define si la letra ingresada es la
// correcta, o si no

// Includes
#include <math.h>
#include <stdio.h>

// Function to validate the entered letter
int ValidateLetterInput(const char letter[]) {
    int lengthArray = sizeof(&letter) / 8;

    // WARNING Este if no funciona correctamente, ya que si se ingresa una letra
    // funciona pero si se ingresa mas de una letra sigue funcionando, lo cual no
    // deberia ser asi ya que solo debe de recibir una letra

    // Check if we have only one letter
    if (lengthArray != 1) {
        return 0;
    } else {
        // Check if the letter is in the correct range (A-Z)
        if ((letter[0] >= 'A' && letter[0] <= 'Z') ||
            (letter[0] >= 'a' && letter[0] <= 'z')) {
            return 1;
        } else {
            return 0;
        }
    }
}
};
```

```

// Function to validate the entered number
int ValidateNumberInput(long int numeroValidar) {
    // Check if the number is in the correct range (0-999999999)
    if (numeroValidar <= 0 || numeroValidar >= pow(10, 8)) {
        return 0;
    } else {
        return 1;
    }
};

// Function to perform the validation of code
int CodeReader(const long int *number, const char *letra) {
    // Array with the correct values
    char validLetter[] = "TRWAGMYFPDXBNJZSQVHLCKE";

    // Calculate the module
    int module = *number % 23;

    // Find the letter in the array
    char letter = validLetter[module];

    // Check if the letter is correct
    if (letter == letra[0]) {
        printf("La letra ingresada es correcta al compararla con el resultado de "
            "la operacion \n");
        return 1;
    } else {
        printf("La letra ingresada no es correcta al compararla con el resultado "
            "de la operacion \n");
        return 0;
    }
}

```

```

// Main Function
int main(void) {
    // Variables
    long int numero;
    char letra[10];

    // Number Input
    printf("Ingresa los numeros correspondientes a tu codigo:");
    scanf("%ld", &numero);

    // Number Validation
    if (ValidateNumberInput(numero)) {
        printf("Tu codigo numerico es correcto \n");
    } else {
        printf("Tu codigo numerico no es correcto\n");
        return 0;
    }

    // Letter Input
    printf("Ingresa la letra correspondiente a tu codigo: ");
    scanf("%s", letra);

    // Letter Validation
    if (ValidateLetterInput(letra)) {
        printf("Tu letra es correcta \n");
    } else {
        printf("Tu letra no es correcta \n");
        return 0;
    }

    // Code Validation
    CodeReader(&numero, letra);
}

```

Se ejecuto el programa usando el código 23 que al realizar la operación de modulo da 0 y esta letra en lo establecido es T y se mandó esa misma letra por lo que el resultado de compararlo es correcto (True) y se realizó una prueba con una letra incorrecta demostrando que al ser una letra distinta daba un resultado de falso.

```
◎ ./codigo.out
Ingresa los numeros correspondientes a tu codigo:23
Tu codigo numerico es correcto
Ingresa la letra correspondiente a tu codigo: R
Tu letra es correcta
La letra ingresada no es correcta al compararla con el resultado de la operacion

3s ◎ ./codigo.out
Ingresa los numeros correspondientes a tu codigo:23
Tu codigo numerico es correcto
Ingresa la letra correspondiente a tu codigo: T
Tu letra es correcta
La letra ingresada es correcta al compararla con el resultado de la operacion
```

Mejoras que se le pueden hacer al código:

- Se puede implementar la librería ctype.h para leer un solo carácter en vez de utilizar un array de caracteres y tomar uno, con esto se retira la necesidad de usar ese array y se pueden realizar las validaciones de una manera mas sencilla, por lo que posteriormente se debe refactorizar la función encargada de validar la letra ingresada.